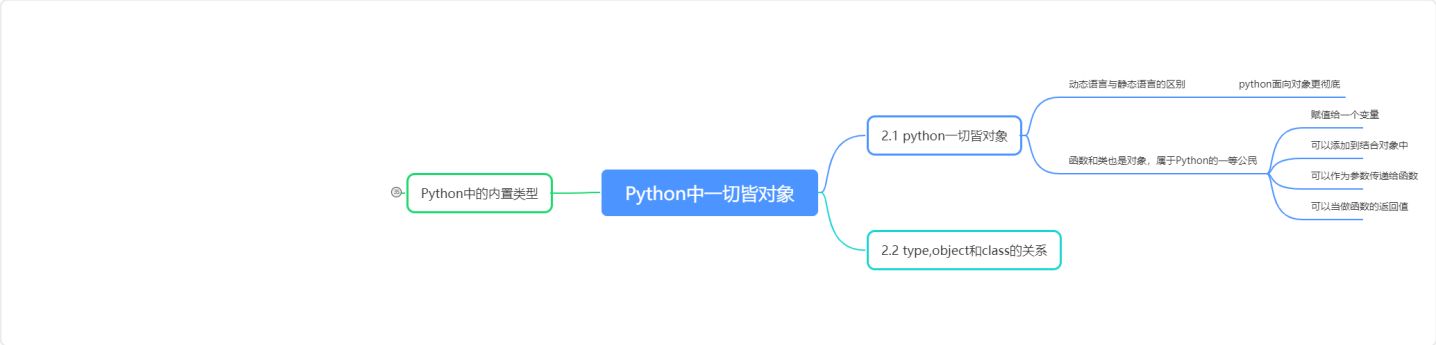


Python中一切皆对象



python中一切皆对象

```
1 def ask(name="bobby"):
2     print(name)
3
4 #函数赋值给变量
5 my_func=ask
6 my_func("bobby")
7
8
9 #类赋值给变量
10 class Person:
11     def __init__(self):
12         print("bobby")
13
14 my_class = Person
15 my_class()
16
17
18 #函数和类添加到集合中
19 obj_list = []
20 obj_list.append(ask)
21 obj_list.append(Person)
22 for item in obj_list:
23     print(item())
24
25 #函数中返回函数
26 def decorator_func():
27     print("dec start")
28     return ask
29
30 ask_func=decorator_func()
31 ask_func("Marry")
32
```

type,object和class的关系

```

1  a=1
2  b="abc"
3  print(type(1))    #int
4  print(type(int))  #type
5  print(type(b))    #str
6  print(type(str))  #type
7
8  #type 1:用于生成类 2:用于返回对象的类型
9  #type 生成int ,int 生成数字  type->class->obj
10
11
12 class Student:
13     pass
14
15 class MyStudent(Student):
16     pass
17
18 stu = Student()
19 print(type(stu))    #<class '__main__.Student'>
20 print(type(Student)) #<class 'type'>
21
22 #object 所有类的最顶层基础类
23 print(int.__bases__)    #(<class 'object'>,)
24 print(str.__bases__)    #(<class 'object'>,)
25 print(Student.__bases__) #(<class 'object'>,)
26 print(MyStudent.__bases__) #(<class '__main__.Student'>)
27
28 #type也是一个类，同时type也是一个对象
29 print(type.__bases__)   #(<class 'object'>,)
30 print(object.__bases__) #()
31 print(type(object))     #<class 'type'>
32

```

魔法函数



什么是魔法函数

```

1 class Compnay(object):
2     def __init__(self, employee_list):
3         self.employy=employee_list
4
5     def __getitem__(self, index):
6         return self.employy[index]
7
8     def __len__(self):
9         return len(self.employy)
10
11
12
13 company = Compnay(["tome", "bob", "jane"])
14 for em in company.employy:
15     print(em)
16 print("-"*80)
17 #加了魔法函数，直接枚举
18 for em in company:
19     print(em)
20
21 print("-"*80)
22 company1=company[:2]
23 for em in company1:
24     print(em)
25
26 print("-"*80)
27 print(len(company1)) #2
28 print(len(company)) #3
29
30 print("-"*80)
31 print(type(company)) #<class '__main__.Compnay'>
32 print(type(company1)) #<class 'list'>

```

魔法函数一览

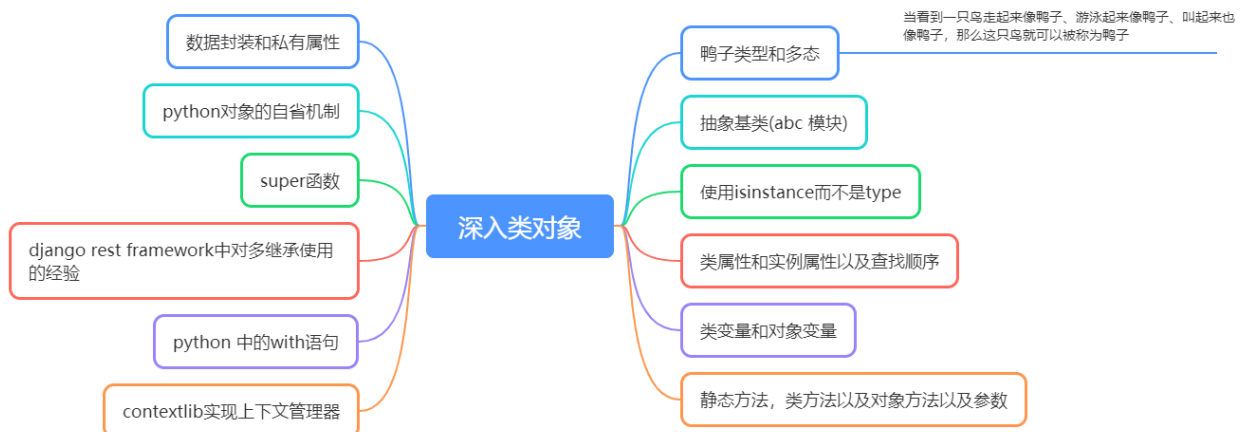
```
1 class Compnay(object):
2     def __init__(self,employee_list):
3         self.employy=employee_list
4
5     def __getitem__(self,index):
6         return self.employy[index]
7
8     def __len__(self):
9         return len(self.employy)
10
11     #对对象进行字符串格式化调用的
12     def __str__(self):
13         return ",".join(self.employy)
14
15     #开发模式调用的
16     def __repr__(self):
17         return ",".join(self.employy)
18
19 company = Compnay(["tome","bob","jane"])
20 print(company)
```

```

1 class MyVector(object):
2     def __init__(self,x,y):
3         self.x=x
4         self.y=y
5
6     def __add__(self,other):
7         re_vector = MyVector(self.x+other.x,self.y+other.y)
8         return re_vector
9
10    def __str__(self):
11        return f"x:{self.x},y:{self.y}"
12
13 first_vec= MyVector(1,2)
14 second_vec= MyVector(2,3)
15 result = first_vec+second_vec
16 print(result)

```

深入类和对象



鸭子类型和多态

```
1 class Cat(object):
2     def say(self):
3         print("i am a cat")
4
5 class Dog(object):
6     def say(self):
7         print("i am a dog")
8
9 class Duct(object):
10    def say(self):
11        print("i am a duct")
12
13 animal = Cat
14 animal().say()
15
16 animal_list=[Cat,Dog,Duct]
17 for animal in animal_list:
18     animal().say()
19
20 a=["bobby1","bobby2"]
21 name_tunple=["bobby3","bobby4"]
22 name_set=set()
23 name_set.add("bobby5")
24 name_set.add("bobby6")
25 a.extend(name_tunple)
26 a.extend(name_set)
27 print(a)
```

抽象基类ABC模块

```
1 #如何检查某个类是否有某种方法
2
3 class Company(object):
4     def __init__(self, employee_list):
5         self.employee=employee_list
6
7     def __len__(self):
8         return len(self.employee)
9
10 company=Company(["bobby1", "bobby2", "bobby3"])
11 if hasattr(company, "__len__"):
12     print(len(company))
13
14 #判断某个对象的类型
15 from collections.abc import Sized
16 print(isinstance(company,Sized))
17
18 #强制某个子类必须实现某些方法
19 #实现了一个web框架，集成cache(redis,cache,memorycache)
20 #需要设计一个抽象基类，指定子类必须实现某些方法
21
22 #实现方法1
23 class CacheBase():
24     def get(self,key,default=None):
25         raise NotImplementedError
26     def set(self,key,value):
27         raise NotImplementedError
28 class RedisCache(CacheBase):
29     pass
30
31 redis_cache=RedisCache()
32 #会抛出异常
33 #redis_cache.set("aa","bb")
34
35 #实现方法2
36 import abc
37 class ABCCacheBase(metaclass=abc.ABCMeta):
```

```
38
39     @abc.abstractmethod
40     def get(self, key, default=None):
41         pass
42
43     @abc.abstractmethod
44     def set(self, key, value):
45         pass
46
47
48 class ABCRedisCache(ABCCacheBase):
49     pass
50
51 #会抛出异常
52 redis_cache=ABCRedisCache()
53
```

使用isinstance而不是type

```
1 class A:
2     pass
3
4 class B(A):
5     pass
6
7 b = B()
8
9 #isinstance 会自动根据继承关系进行判断
10 print(isinstance(b,B)) #True
11 print(isinstance(b,A)) #True
12
13
14 #is 底层是根据id()来判断
15 #== 判断值是否相等
16 print(type(b) is B) #True
17 print(type(b) == B) #True
18
19 print(type(b) is A) #False
20 print(type(b) == A) #False
21
22
```

类变量和实例变量

```
1 class A:
2
3     aa=1 #类变量
4
5     def __init__(self,x,y):
6         self.x=x    #实例变量
7         self.y=y    #实例变量
8
9 a=A(2,3)
10
11 #2 3 1 1 通过实例访问：先查找对象变量再查找类变量。
12 #类变量可以通过类名访问,通过类名访问不会查找实例变量
13 print(a.x,a.y,a.aa,A.aa)
14
15
16 #修改类变量的值
17 A.aa=11
18 print(a.aa) #11 通过实例变量访问了类变量aa的值,为11
19
20 #相当于给a这个实例添加了一个实例变量aa,值为100, 不会改变类变量A.aa的值
21 a.aa=100
22 print(A.aa) #11
23
24 b=A(3,5)
25
26 #类变量共享
27 print(b.aa) #11
```

类和实例的查找顺序

```
1 class D(object):
2     pass
3
4 class C(D):
5     pass
6
7 class B(D):
8     pass
9
10 class A(B, C):
11     pass
12
13
14 #(<class '__main__.A'>, <class '__main__.B'>, <class '__main__.C'>, <class
    '__main__.D'>, <class 'object'>)
15 print(A.__mro__)
16
17
18 class D1:
19     pass
20
21 class E1:
22     pass
23
24 class C1(E1):
25     pass
26
27 class B1(D1):
28     pass
29
30 class A1(B1, C1):
31     pass
32
33 #(<class '__main__.A1'>, <class '__main__.B1'>, <class '__main__.D1'>, <class
    '__main__.C1'>, <class '__main__.E1'>, <class 'object'>)
34 print(A1.__mro__)
35
36
```

```
37 class A2:
38     name="A"
39
40     def __init__(self):
41         self.name="obj"
42
43 a=A2()
44
45 #输出:obj 先查找实例属性，再查找类属性
46 print(a.name)
```

静态方法类方法对象方法以及参数

```

1  class Date:
2
3      def __init__(self,year,month,day):
4          self.year=year
5          self.month=month
6          self.day=day
7
8      def __str__(self):
9          return "
{year}/{month}/{day}".format(year=self.year,month=self.month,day=self.day)
10
11     #实例方法
12     def tomorrow(self):
13         self.day+=1
14
15     #静态方法
16     @staticmethod
17     def parse_from_string(date_str):
18         year,month,day=tuple(date_str.split("-"))
19         return Date(int(year),int(month),int(day))
20
21     #类方法
22     @classmethod
23     def from_string(cls,date_str):
24         year, month, day = tuple(date_str.split("-"))
25         return cls(int(year), int(month), int(day))
26
27 if __name__ == '__main__':
28     new_day = Date(2018,12,31)
29     print(new_day)
30
31     new_day.tomorrow()
32     print(new_day)
33
34     new_day=Date.parse_from_string("2018-12-31")
35     print(new_day)
36
37     new_day = Date.from_string("2018-11-31")

```

```
38     print(new_day)
```

```
39
```

数据封装和私有属性

```
1  class User:
2      def __init__(self,birthyear):
3          #私有属性以双下划线开始,外部不可访问
4          #python将该变量名称重写为_User__birthday,该名称可以在外部访问
5          #python的私有性并非绝对安全
6          self.__birthyear=birthyear
7
8      def get_age(self):
9          return 2018-self.__birthyear
10
11
12  if __name__ == '__main__':
13      user=User(2012)
14      print(user.get_age())
```

自省机制

```

1 class Person:
2     """
3     人
4     """
5     name="user"
6
7 class Student(Person):
8     def __init__(self,school_name):
9         self.school_name = school_name
10
11
12 if __name__ == '__main__':
13     user = Student("user")
14     #通过__dict__ 查询属性
15     print(user.__dict__) #{'school_name': 'user'}
16     #name属于Person类
17     print(user.name)      #user
18     #user
19     #{'__module__': '__main__', '__doc__': '\n    人\n    ', 'name': 'user',
20     '__dict__': <attribute '__dict__' of 'Person' objects>, '__weakref__': <attribute
21     '__weakref__' of 'Person' objects>}
22     print(Person.__dict__)
23     user.__dict__["school_addr"]="北京市"
24     print(user.school_addr) #北京市
25     #['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
26     '__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__',
27     '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
28     '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
29     '__subclasshook__', '__weakref__', 'name', 'school_addr', 'school_name']
30     print(dir(user))
31

```

super

```
1 class A:
2     def __init__(self):
3         print("A")
4
5 class B(A):
6     def __init__(self):
7         print("B")
8         #执行MRO顺序的下一个的构造方法
9         super().__init__()
```

```
1 class A:
2     def __init__(self):
3         print("A")
4
5 class B(A):
6     def __init__(self):
7         print("B")
8         super().__init__()
9
10 class C(A):
11     def __init__(self):
12         print("C")
13         super().__init__()
14
15 class D(B,C):
16     def __init__(self):
17         print("D")
18         super().__init__()
19
20
21 d=D() #D B C A
22 #(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class
    '__main__.A'>, <class 'object'>))
23 print(D.__mro__)
```

- 1 #mixin模式特点
- 2 #1.Mixin类功能单一
- 3 #2.不和基类关联，可以和任意基类组合，基类可以不和mixin关联就能初始化成功
- 4 #3.在mixin中不要使用super这种用法

```
1 import time
2 import logging
3
4 # Mixin 1: 日志能力
5 class LogMixin:
6     def log_info(self, msg):
7         logging.info(f"[{self.__class__.__name__}] {msg}")
8
9 # Mixin 2: 缓存能力（带过期时间）
10 class CacheMixin:
11     _cache = {}
12
13     def set_cache(self, key, value, ttl=60):
14         self._cache[key] = (value, time.time() + ttl)
15
16     def get_cache(self, key):
17         if key in self._cache:
18             value, expire = self._cache[key]
19             if time.time() < expire:
20                 return value
21             del self._cache[key]
22         return None
23
24 # 业务类：同时获得日志和缓存能力
25 class DataService(LogMixin, CacheMixin):
26     def fetch_data(self, key):
27         # 先查缓存
28         cached = self.get_cache(key)
29         if cached:
30             self.log_info(f"Hit cache for {key}")
31             return cached
32
33         # 模拟查询数据库
34         self.log_info(f"Query DB for {key}")
35         data = f"data_{key}"
36         self.set_cache(key, data, ttl=30)
37         return data
38
```

```
39 # 使用
40 service = DataService()
41 print(service.fetch_data("user_1")) # 第一次查 DB
42 print(service.fetch_data("user_1")) # 第二次走缓存 + 日志
```

上下文管理

```
1 #上下文管理器
2 class Sample:
3     def __enter__(self):
4         #获取资源
5         print("enter")
6         return self
7
8     def __exit__(self, exc_type, exc_val, exc_tb):
9         #释放资源
10        print("exit")
11
12    def do_something(self):
13        print("do_something")
14
15 if __name__ == '__main__':
16     with Sample() as sample:
17         sample.do_something()
```

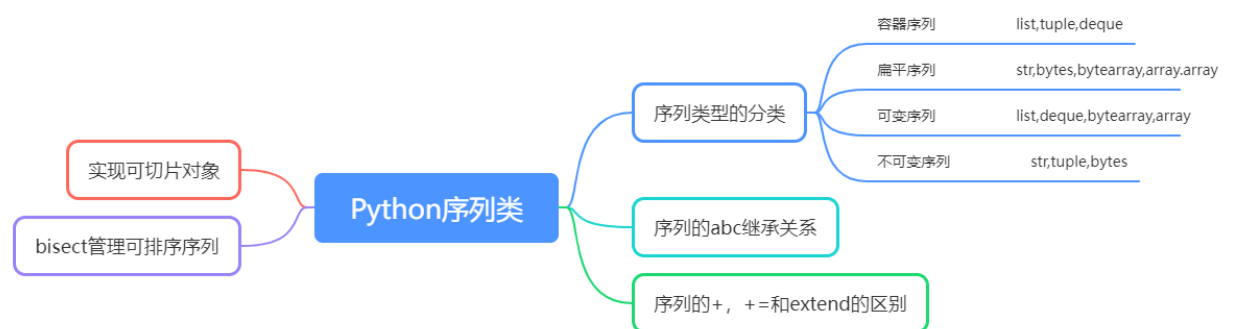
contextlib上下文管理

```

1 import contextlib
2
3 @contextlib.contextmanager
4 def file_open(file_name):
5     print("file open")
6     yield {}
7     print("file end")
8
9 with file_open("abcd.txt") as f:
10    print("file process")
11    print(f)
12
13 #输出内容如下:
14 #file open
15 #file process
16 #{}
17 #file end

```

Python序列类



容器序列存放任意类型数据

```
1 my_list=[]
2 my_list.append(1)
3 my_list.append("a")
```

扩展操作

```
1 a=[1,2]
2 c=a+[3,4] # 产生一个新的list,不能改为元组
3 print(c) # [1,2,3,4]
4 a+= [3,4] # a里面添加元素,不产生新的list,可以改为元组,通过__iadd__魔法函数来实现
5 print(a) # [1,2,3,4]
6 a.extend(range(3)) #原对象扩展元素,不产生新对象
7 a.append([1,2]) #将数组看成一个值
8 a.append((3,4)) #将元组看成一个值
9 print(a)
```

切片操作

```

1 #模式[start:end:step]
2 """
3     其中，第一个数字start表示切片开始位置，默认为0；
4     第二个数字end表示切片截止(但不包含)位置(默认为列表长度)；
5     第三个数字step表示切片的步长(默认为1)。
6     当start为0时可以省略，当end为列表长度时可以省略，
7     当step为1时可以省略，并且省略步长时可以同时省略最后一个冒号。
8     另外，当step为负整数时，表示反向切片，这时start应该比end的值要大才行。
9 """
10 aList =[3,4,5,6,7,9,11,13,15,17]
11 aList[:]          #返回包含原列表中所有元素的新列表
12 aList[::-1]       #返回包含原列表中所有元素的逆序列表
13 print(aList[::2])  #隔一个取一个，获取偶数位置的元素
14 print(aList[1::2]) #隔一个取一个，获取奇数位置的元素
15 print(aList[3:6])  #指定切片的开始和结束位置
16 aList[0:100]       #切片结束位置大于列表长度时，从列表尾部截断
17 aList[100:]        #切片开始位置大于列表长度时，返回空列表
18 aList[len(aList):]=[9] #在列表尾部增加元素
19 aList[:0]=[1,2]     #在列表头部插入元素
20 aList[3:3] = [4]    #在列表中间位置插入元素
21 aList[:3]=[1,2]     #替换列表元素，等号两边的列表长度相等
22 aList[3:]=[4,5,6]   #等号两边的列表长度也可以不相等
23 aList[::2]=[0]*3     #隔一个修改一个
24 aList[::2]=['a','b','c'] #隔一个修改一个
25 #aList[::2]=[1,2]    #左侧切片不连续，等号两边列表长度必须相等，否则会引引起异常
26 aList[:3]=[]        #删除列表中前3个元素
27 del aList[:3]       #删除前3个元素
28 del aList[::2]      #删除元素不连续，隔一个删一个

```

实现可切片对象

```

1 class Group(object):
2     #支持切片操作，切片以后还是Group
3     def __init__(self,group_name,company_name,staffs):
4         self.group_name = group_name
5         self.company_name = company_name
6         self.staffs = staffs
7
8     def __reversed__(self):
9         self.staffs.reverse()
10
11    def __getitem__(self, item):
12        import numbers
13        cls = type(self)
14        if isinstance(item,slice):
15            return cls(self.group_name,self.company_name,self.staffs[item])
16        elif isinstance(item,numbers.Integral):
17            return cls(self.group_name,self.company_name,[self.staffs[item]])
18
19    def __len__(self):
20        return len(self.staffs)
21
22    def __iter__(self):
23        return iter(self.staffs)
24
25    def __contains__(self, item):
26        if item in self.staffs:
27            return True
28        else:
29            return False
30
31    if __name__ == '__main__':
32        staffs=["bobby1","immoc","bobby2","bobby3"]
33        group =Group(company_name="imoooc",group_name="user",staffs=staffs)
34        sub_group=group[:2]
35        print(type(sub_group))
36        sub_group=group[1]
37        print(type(sub_group))
38        if "bobby1" in group:

```

```
39     print("Yes")
40     print(len(group))
41     for user in group:
42         print(user)
```

bisect维护已排序序列

```
1  import bisect
2  from collections import deque
3
4  #用来处理已排序的序列，用来维持已排序的序列，升序
5  #二分查找
6
7  inter_list=[]
8  inter_list=deque()
9
10 bisect.insort(inter_list,2)
11 bisect.insort(inter_list,3)
12 bisect.insort(inter_list,5)
13 bisect.insort(inter_list,1)
14 bisect.insort(inter_list,6)
15 #[1, 2, 3, 5, 6]
16 print(inter_list)
17 #判断要插入元素的位置
18 print(bisect.bisect_left(inter_list,3))
19 print(bisect.bisect_right(inter_list,3))
20 print(bisect.bisect(inter_list,3))
```

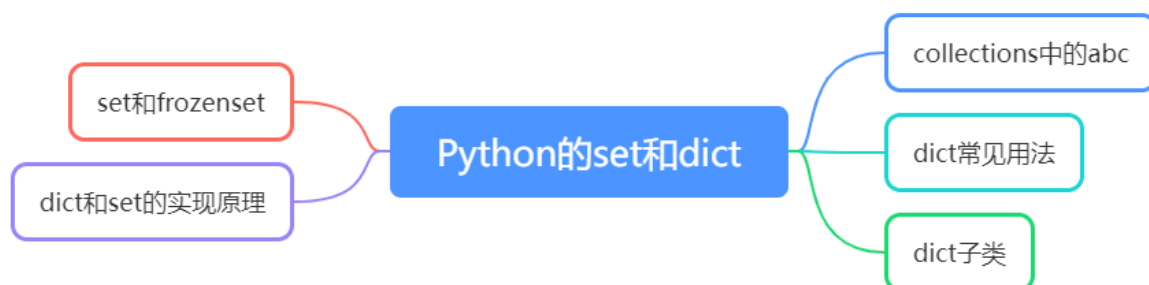
Array

```
1 import array
2
3 #array 数组，连续的内存空间
4 #只能存放指定类型的数据类型
5 #my_array = array.array('i',[1,2,3,4,5,6,7,8,9])
6 my_array = array.array('i')
7 my_array.append(1)
8 #my_array.append("abcd") #报错
```

列表推导式、生成器表达式、字典推导式

```
1 #列表推导式
2 #提取1-20之间的奇数
3 l1=[ i for i in range(0,21) if i%2==1 ]
4 print(l1)
5
6 #逻辑复杂的情况
7 def num_pf(num):
8     return num*num
9
10 l2=[ num_pf(i) for i in range(0,21) if i%2==1 ]
11 print(l2)
12
13
14 #生成器表达式
15 gen1=(i for i in range(0,21) if i%2==1 )
16 #<class 'generator'>
17 print(type(gen1))
18 #枚举生成器值
19 for item in gen1:
20     print(item)
21
22 #生成器转list
23 l3 = list(gen1)
24 print(type(l3))
25
26
27 #字典推导式
28 my_dict={"bobby1":22,"bobby2":23,"imooc.com":5}
29 reversed_dict={value:key for key,value in my_dict.items()}
30 print(reversed_dict)
31
32 #集合推导式
33 my_set={key for key,value in my_dict.items()}
34 my_set2=set(my_dict.keys())
35 print(my_set)
36 print(my_set2)
37 print(type(my_set))
```

Python的Set和Dict



dict对象属于MutableMapping

```
1 from collections.abc import Mapping,MutableMapping
2
3 a={}
4 #True dict对象属于MutableMapping
5 print(isinstance(a,MutableMapping))
```

dict常用操作

清空

```
1 a={
2     "bobby1":{"company":"imoc"} ,
3     "bobby2":{"company":"imoc2"}
4 }
5
6 #清空
7 #a.clear()
8
```

浅拷贝

```
1 #浅拷贝
2 new_dict=a.copy()
3 print(new_dict)
4 new_dict["bobby1"]["company"]="imocx"
5 #输出:imocx
6 print(a["bobby1"]["company"])
7
```

深拷贝

```
1 #深拷贝
2 import copy
3 a={
4     "bobby1":{"company":"imoc"} ,
5     "bobby2":{"company":"imoc2"}
6 }
7 new_dict2=copy.deepcopy(a)
8 new_dict["bobby1"]["company"]="imocx"
9 #输出:imoc
10 print(a["bobby1"]["company"])
11
```

fromkeys构建

```
1 #fromkeys构建Dict
2 l1=["bobby1","bobby2"]
3 new_dict3=dict.fromkeys(l1,{"company":"immoc"})
4 print(new_dict3)
5
```

get方法

```
1 #key不存在，会抛KeyError异常
2 #new_dict3["notakey"]
3 #通过get方法,在可以不存在的时候，可以返回默认值
4 value=new_dict3.get("notakey","defaultval")
5 print(value)
```

items方法枚举dict

```
1 for key,value in new_dict3.items():
2     print(key,value)
```

setdefault方法

```
1
2 dict4={"bobby2":"value2"}
3 #setdefault
4 value=dict4.setdefault("bobby2","immoc2")
5 #value2,先返回key对应的原始值，再设置新值。如果key不存在，返回设置的新值。
6 print(value)
7 value=dict4.setdefault("bobby","immoc")
8 #immoc
9 print(value)
```

update方法合并

```
1 #update Dict合并
2 d1={"k1":"v1"}
3 #d1.update(k1="v11",k2="v2")
4 #print(d1)
5 #d1.update({"k1":"v11","k2":"v2"})
6 #print(d1)
7 #d1.update([("k1","v11"),("k2","v2")])
8 #print(d1)
9 d1.update((( "k1","v11"),("k2","v2"))))
10 print(d1)
```

dict的子类

不正确的继承

```
1 class MyDict(dict):
2     def __setitem__(self, key, value):
3         super().__setitem__(key,value*2)
4
5 my_dict=MyDict(one=1)
6 #{'one': 1}
7 print(my_dict)
8 my_dict["one"]=1
9 #{'one': 2}
10 print(my_dict)
```

正确的继承(UserDict)

```
1 from collections import UserDict
2 class MyDict(UserDict):
3     def __setitem__(self, key, value):
4         super().__setitem__(key, value*2)
5
6 my_dict=MyDict(one=1)
7 #{'one': 2}
8 print(my_dict)
```

defaultdict

```
1 from collections import defaultdict
2 my_dict=defaultdict(dict)
3 my_value=my_dict["bobby"]
4 print(my_value)
```

set 和 frozenset

```
1 #集合,无序,不重复
2 #set集合,构建接受序列
3 s=set("abcdee")
4 s=set(['a','b','c','d','e','e'])
5 s={'a','b','c','d','e','e'}
6 print(s)
7 #添加数据
8 s1=set()
9 s1.add("a")
10 s1.add("b")
11 s1.add("c")
12
13 #update更新集合
14 s2={"b","d","f"}
15 s1.update(s2)
16 print(s1)
17
18 #集合操作
19 #差集 s1-s2 返回新对象 -
20 sn=s1.difference(s2)
21
22 #并集
23 sn=s1.union(s2)
24 sn=s1|s2
25
26
27 #交集
28 sn=s1.intersection(s2)
29 sn=s1&s2
30
31
32 #判断存在性
33
34 if "a" in sn:
35     print("yes")
36
37 #子集判断
```

```
38 s1.issubset(s2)
39 #超集判断
40 s1.issubset(s2)
41
42 #frozenset(不可变集合),可以作为dict的key
43 s=frozenset("abcdee")
```

实现原理

dict查找的性能远远大于list。

在List中随着list数据的增大 查找时间会增大。

在dict中查找元素不会随着dict的增大而增大。

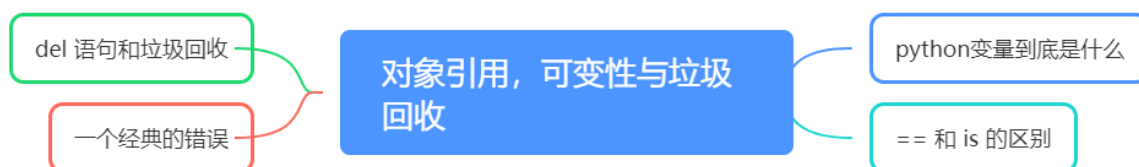
1:dict的key或者set的值 都必须是可以hash的。不可变对象 都是可hash的, str, frozenset, tuple, 自己实现的类 hash。

2:dict的内存花销大, 但是查询速度快, 自定义的对象 或者python内部的对象都是用dict包装的。

3:dict的存储顺序和元素添加顺序有关。

4:添加数据有可能改变已有数据的顺序。

对象引用, 可变性与垃圾回收



变量的实质

```
1 #python和java中的变量本质不一样，python的变量实质上是一个指针 int str 便利贴
2 a=1
3 a= "abc"
4 #1 a贴在1上面
5 #2 先生成对象 然后贴便利贴
6 a=[1,2,3]
7 b=a
8 print (id(a), id(b))
9 print (a is b)I
10 # b.append(4)
11 # print (a)
12
```

is 和 == 的区别

```
1  a=[1,2,3,4]
2  b=[1,2,3,4]
3  #False
4  print(id(a)==id(b))
5  #False
6  print(a is b)
7  #True 调用eq魔法函数判断内容是否相等
8  print(a==b)
9
10
11 a=[1,2,3,4]
12 b=a
13 #a和b是同一个对象
14 #True
15 print(id(a)==id(b))
16 #True
17 print(a is b)
18
19
20 #对象缓存
21 a=1
22 b=1
23 #True
24 print(id(a)==id(b))
25 #True
26 print(a is b)
27 a="abc"
28 b="abc"
29 #True
30 print(id(a)==id(b))
31 #True
32 print(a is b)
33
34
35
36 #对象类型判断
37 class People:
38     pass
```

```
39
40 person = People()
41 if type(person) is People:
42     pass
43
44 if isinstance(person, People):
45     pass
```

del语句和垃圾回收

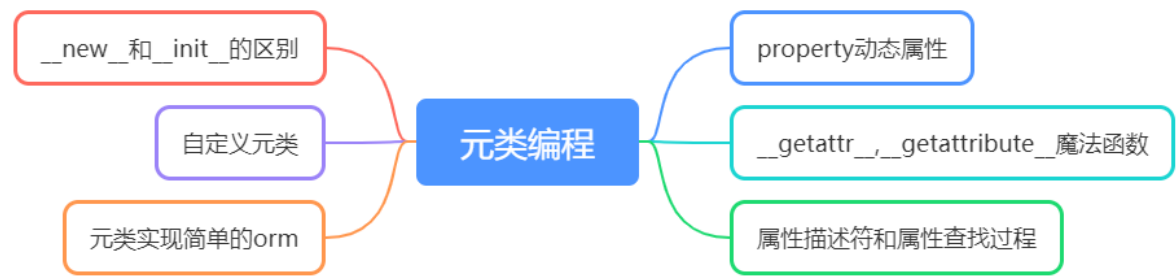
```
1 #python垃圾回收采用引用计数法
2 a=object()
3 b=a
4 del a
5 print(b) #正常打印
6 #print(a) #变量未定义异常，对象并未真正回收
7
8
9 class A:
10     def __del__(self):
11         print("A is delete")
12
13 a1=A()
14 a1.age=3
15 a2=a1
16 del a1 # 打印,但未回收对象 。A is delete
17 print(a2.age) #打印3
```

一个经典的错误

```
1 def add(a,b):
2     a+=b
3     return a
4
5 class Company:
6     def __init__(self,name,staffs=[]):
7         self.name=name
8         self.staffs=staffs
9
10    def add(self,staff_name):
11        self.staffs.append(staff_name)
12
13    def remove(self,staff_name):
14        self.staffs.remove(staff_name)
15
16
17
18 def test_001():
19     a = 1
20     b = 2
21     c = add(a, b)
22     # a=1 b=2 c=3
23     print("a=", a, " b=", b, " c=", c, sep="")
24     a = [1, 2]
25     b = [3, 4]
26     c = add(a, b)
27     # a=[1, 2, 3, 4] b=[3, 4] c=[1, 2, 3, 4]
28     print("a=", a, " b=", b, " c=", c, sep="")
29
30     a = (1, 2)
31     b = (3, 4)
32     c = add(a, b)
33     # a=(1, 2) b=(3, 4) c=(1, 2, 3, 4)
34     print("a=", a, " b=", b, " c=", c, sep="")
35
36
37 def test_002():
```

```
38     com1=Company("com1",["bobby1","bobby2"])
39     com1.add("bobby3")
40     com1.remove("bobby1")
41     print(com1.staffs) #['bobby2', 'bobby3']
42
43     com2=Company("com2")
44     com2.add("bobby")
45     print(com2.staffs) #bobby
46
47     com3=Company("com3")
48     com3.add("bobby5")
49     #com2 和 com3共用一个list,两者之间会相互影响
50     print(com2.staffs) #['bobby', 'bobby5']
51     print(com3.staffs) #['bobby', 'bobby5']
52
53     print(com2.staffs is com3.staffs) #True
54
55     #(['bobby', 'bobby5'],) com2,com3共用的list对象
56     print(Company.__init__.__defaults__)
57
58
59
60 if __name__ == '__main__':
61     test_001()
62     test_002()
```

元类编程



property动态属性

```

1 from datetime import date,datetime
2
3 class User:
4     def __init__(self,name,birthday):
5         self.name=name
6         self.birthday=birthday
7         self._age=0
8
9     def get_age(self):
10         return datetime.now().year - self.birthday.year
11
12     @property
13     def age(self):
14         return datetime.now().year - self.birthday.year
15
16     @age.setter
17     def age(self,value):
18         self._age=value
19
20 if __name__ == '__main__':
21     user = User("bobby",date(year=1987,month=1,day=1))
22     user.age=30
23     print(user.age)
24     print(user._age)

```

__getattr__和__get_attribute__魔法函数

```

1  #__getattr__ 查找不到属性的时候调用
2  from datetime import date
3  class User:
4      def __init__(self,name,birthday):
5          self.name = name
6          self.birthday = birthday
7
8      def __getattr__(self, item):
9          print("not find attr")
10         return "not find attribute"
11
12     def __getattribute__(self, item):
13         #无条件访问该魔法函数
14         print("__getattribute__ invoke !")
15
16 if __name__ == '__main__':
17     user = User("bobby",date(year=1989,month=8,day=1))
18     #没有age属性,调用__getattr__魔法函数,打印: not find attr ,返回值 "not find attribute"
19     print(user.age)
20     #能找到name,不会调用__getattr__魔法函数
21     print(user.name)

```

如果user是某个类的实例,那么user.age(以及等价的getattr(user,'age'))首先调用__getattribute__。如果类定义了__getattr__方法,那么在__getattribute__抛出 AttributeError的时候就会调用到__getattr__,而对于描述符(get)的调用,则是发生在__getattribute__内部的。user==User(),那么user.age 顺序如下: (1)如果“age”是出现在User或其基类的__dict__中,且age是data descriptor, 那么调用其__get__方法, 否则(2)如果“age”出现在obj的__dict__中,那么直接返回obj.dict['age'],否则(3)如果“age”出现在User或其基类的__dict__中(3.1)如果age是非-data descriptor,那么调用其__get__方法,否则(3.2)返回__dict__['age']

(4)如果User有__getattr__方法,调用__getattr__方法,否则

(5)抛出AttributeError

new和init

```
1 # new 是用来控制对象的生成过程，在对象生成之前
2 # init是用来完善对象的
3 # 如果new方法不返回对象， 则不会调用init函数
4 class User:
5     def __new__(cls,user):
6         #定义类的生成过程
7         print("in new")
8         return super().__new__(cls)
9
10    def __init__(self,name):
11        #定义对象的构建过程
12        print("in init")
13        #在对象上定义name属性
14        self.name = name
15        pass
16
17 if __name__ == '__main__':
18     user = User("bobby")
19
```

自定义元类

动态创建类

```
1 #类也是对象，type创建类的类
2 def create_class(name):
3     if name == "user":
4         class User:
5             def __str__(self):
6                 return "user"
7         return User
8     elif name == "company" :
9         class Company:
10             def __str__(self):
11                 return "company"
12         return Company
13
14
15 if __name__ == '__main__':
16     MyClass = create_class('user')
17     my_obj = MyClass()
18     #<class 'type'>
19     print(type(MyClass))
20     #<class '__main__.create_class.<locals>.User'>
21     print(type(my_obj))
22
```

通过type创建类

```
1 class BaseClass:
2     def answer(self):
3         return "i am baseclass"
4
5 def say(self):
6     return self.name
7
8 if __name__ == "__main__":
9     # type动态创建类
10    User = type("User", (BaseClass,), {"name": "bobby", "say": say})
11    my_obj = User()
12    #bobby
13    print(my_obj.name)
14    #bobby
15    print(my_obj.say())
16    #i am baseclass
17    print(my_obj.answer())
18
```

MetaClass的使用

```
1 #什么是元类，元类是创建类的类,对象<-class(对象)<-type
2
3 class MetaClass(type):
4     def __new__(cls, *args,**kwargs):
5         return super().__new__(cls,*args,**kwargs)
6
7 class User(metaclass=MetaClass):
8     def __init__(self,name):
9         self.name = name
10
11     def __str__(self):
12         return "user"
13
14 if __name__ == '__main__':
15     my_obj = User(name="bobby")
16     print(my_obj)
```

orm实现案例

```
1 import numbers
2
3 class Field:
4     pass
5
6 class IntField(Field):
7     def __init__(self,db_column,min_value=None,max_value=None):
8         self._value = None
9         self.min_value = min_value
10        self.max_value = max_value
11        self.db_column = db_column
12        if min_value is not None :
13            if not isinstance(min_value,numbers.Integral):
14                raise ValueError("min_value must be int")
15
16        if max_value is not None:
17            if not isinstance(max_value, numbers.Integral):
18                raise ValueError("max_value must be int")
19
20        if min_value is not None and max_value is not None:
21            if min_value > max_value:
22                raise ValueError("min_value must be less than max_value")
23
24    def __get__(self, instance, owner):
25        return self._value
26
27    def __set__(self, instance, value):
28        if not isinstance(value,numbers.Integral):
29            raise ValueError("value must be int")
30        if value < self.min_value:
31            raise ValueError("value must be less than min_value")
32        if value > self.max_value:
33            raise ValueError("value must be greater than min_value")
34        self._value = value
35
36
37 class CharField(Field):
38     def __init__(self,db_column,max_length=None):
```

```

39         self._value = None
40         self.db_column = db_column
41         if max_length is None:
42             raise ValueError("max_length is required")
43         self.max_length = max_length
44
45     def __get__(self, instance, owner):
46         return self._value
47
48     def __set__(self, instance, value):
49         if not isinstance(value, str):
50             raise ValueError("string value needed")
51         if len(value) > self.max_length:
52             raise ValueError("value length excess max_length")
53         self._value = value
54
55
56 class ModelMetaClass(type):
57     def __new__(cls, name, bases, attrs, **kwargs):
58         if name == "BaseModel":
59             return super().__new__(cls, name, bases, attrs, **kwargs)
60
61         fields = {}
62         for key, value in attrs.items():
63             if isinstance(value, Field):
64                 fields[key] = value
65
66         attrs_meta = attrs.get("Meta", None)
67         _meta = {}
68         db_table = name.lower()
69         if attrs_meta is not None:
70             table = getattr(attrs_meta, "db_table", None)
71             if table is not None:
72                 db_table = table
73         _meta["db_table"] = db_table
74         attrs["_meta"] = _meta
75         attrs["fields"] = fields
76         del attrs["Meta"]
77         return super().__new__(cls, name, bases, attrs, **kwargs)

```

```

78
79
80 class BaseModel(metaclass=ModelMetaClass):
81     def __init__(self,*args,**kwargs):
82         for key,value in kwargs.items():
83             setattr(self,key,value)
84         return super().__init__()
85
86     def save(self):
87         fields=[]
88         values=[]
89         for key,value in self.fields.items():
90             db_column = value.db_column
91             if db_column is None:
92                 db_column = key
93             fields.append(db_column)
94             value=getattr(self,key)
95             values.append(str(value))
96         sql = "insert into {db_table}({fields}) values
97         ({values})".format(db_table=self.db_table,fields=",".join(fields),values=",".join(values))
98
99         pass
100
101 class User(BaseModel):
102     name=CharField(db_column="name",max_length=10)
103     age=IntegerField(db_column="age",min_value=0,max_value=100)

```

迭代器和生成器



迭代器协议

```
1 #什么是迭代协议(可迭代类型[Iterable], 迭代器[Iterator])
2 #迭代器是什么? 迭代器是访问集合内元素的一种方式, 一般用来遍历数据
3 #迭代器和以下标的访问方式不一样, 迭代器是不能返回的, 迭代器提供了一种惰性方式数据的方式
4 #[] list , __iter__
5
6 from collections.abc import Iterable,Iterator
7 a=[1,2]
8 #list是一个可迭代的对象, 但不是迭代器
9 print(isinstance(a,Iterable)) #True
10 print(isinstance(a,Iterator)) #False
11 #通过list生成一个iterator
12 iterator = iter(a)
13 print(isinstance(iterator,Iterator)) #True
```

实现迭代器

```

1 class Company(object):
2     def __init__(self, employee_list):
3         self.employee = employee_list
4
5     def __iter__(self):
6         return MyIteraotr(self.employee)
7
8     def __getitem__(self, item):
9         return self.employee[item]
10
11
12 class MyIteraotr(Iterator):
13     def __init__(self, employee_list):
14         self.iter_list=employee_list
15         self.index=0
16     def __next__(self):
17         try:
18             word=self.iter_list[self.in ex]
19         except IndexError:
20             raise StopIteration
21         self.index+=1
22         return word
23
24
25 if __name__=="__main__":
26     company = Company(["tom","bob","jane"])
27     #有__iter__先调用，没有再考虑__get_item__
28     #iter_company=iter(company)
29     for item in company:
30         print(item)

```

生成器的使用

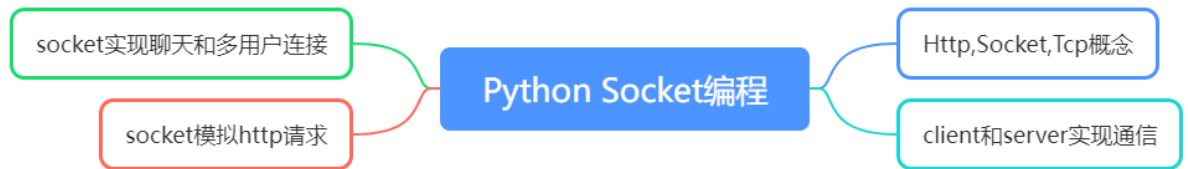
```
1 #函数里面有yield关键字就是生成器函数
2 #为延迟求值提供了可能
3 def gen_func():
4     yield 1
5     yield 2
6     yield 3
7
8 def func():
9     return 1
10
11 def fib(index):
12     if index<=2:
13         return 1
14     else:
15         return fib(index-1)+fib(index-2)
16
17 def fib2(index):
18     re_list=[]
19     n,a,b=0,0,1
20     while n<index:
21         re_list.append(b)
22         a,b=b,a+b
23         n+=1
24     return re_list
25
26 def gen_fib(index):
27     n,a,b=0,0,1
28     while n<index:
29         yield b
30         a,b=b,a+b
31         n+=1
32
33
34 if __name__ == '__main__':
35     #生成器对象，python编译字节码的时候就产生了
36     gen = gen_func()
37     #<generator object gen_func at 0x000001E78B196D50>
38     print(gen)
```

```
39     #访问生成器对象返回内容
40     for val in gen:
41         print(val)
42
43     re = func()
44     # 1
45     print(re)
46
47     for val in gen_fib(10):
48         print(val,end=",")
49
50
```

生成器读取大文件

```
1  from idlelib.debugobj import myrepr
2
3
4  def myradlines(f,newline):
5      buf=""
6      while True:
7          while newline in buf:
8              pos = buf.index(newline)
9              yield buf[:pos]
10             buf = buf[pos + len(newline):]
11             chunk = f.read(4096)
12             if not chunk:
13                 yield buf
14                 break
15             buf += chunk
16
17  if __name__ == '__main__':
18      with open("D://tmp//code.txt") as f:
19          for line in myradlines(f,"{|}"):
20              print(line)
```

Python Socket



五层网络模型

client与server通讯实现

Socket客户端

```
1 import socket
2
3 if __name__ == '__main__':
4     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5     client.connect(('127.0.0.1', 8000))
6     client.send("bobby".encode("utf8"))
7     data = client.recv(1024)
8     print("="*80)
9     print(data.decode("utf8"))
10    print("=" * 80)
11    client.close()
12
```

Socket服务端

```
1 import socket
2
3 if __name__ == '__main__':
4     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5     server.bind(('0.0.0.0', 8000))
6     server.listen()
7     sock, addr = server.accept()
8     #获取从客户端发的数据,一次1K
9     data = sock.recv(1024)
10    str_data = data.decode("utf8")
11    print("="*80)
12    print(str_data)
13    print("=" * 80)
14    sock.send(f"hello {str_data}".encode("utf8"))
15    server.close()
16    sock.close()
17
```

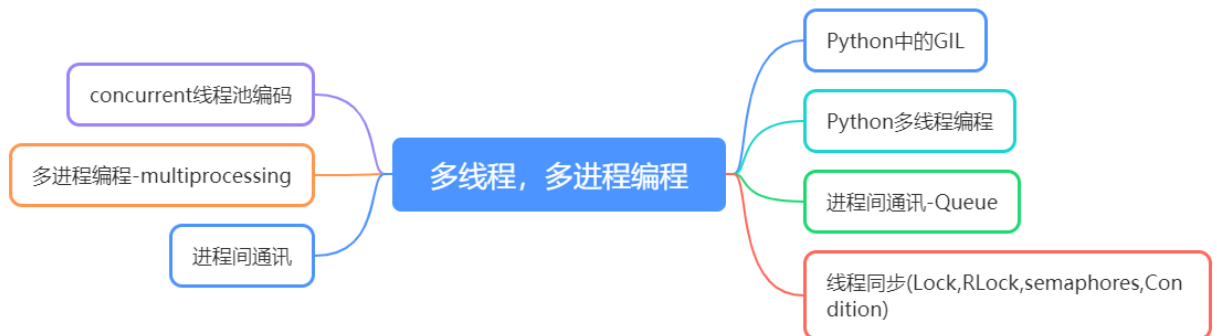
多客户端聊天实现

```
1 import socket
2 import threading
3
4
5 def handle_socket(sock, addr):
6     while True:
7         data = sock.recv(1024)
8         str_data = data.decode("utf8")
9         print("=" * 80)
10        print(str_data)
11        print("=" * 80)
12        re_data = input()
13        sock.send(re_data.encode("utf8"))
14
15 if __name__ == '__main__':
16     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
17     server.bind(('0.0.0.0', 8000))
18     server.listen()
19     while True:
20         sock, addr = server.accept()
21         # 获取从客户端发的数据,一次1K
22         client_thread = threading.Thread(target=handle_socket, args=(sock, addr))
23         client_thread.start()
24     server.close()
25     sock.close()
26
```

实现HTTP通讯

```
1 import socket
2 from urllib.parse import urlparse
3
4 def get_url(url):
5     url = urlparse(url)
6     host=""
7     port=80
8     path=url.path
9     if url.netloc.count(":")>0 :
10         seg=url.netloc.split(":")
11         host=seg[0]
12         port=int(seg[1])
13     else:
14         host=url.netloc
15
16     if path == "":
17         path = "/"
18
19     print(host)
20     print(port)
21     print(path)
22     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
23     client.connect((host,port))
24     client.send("GET {} HTTP/1.1\r\nHost:
25 {} \r\nConnection:close\r\n\r\n".format(path,host).encode("utf8"))
26
27     data = b""
28     while True:
29         d = client.recv(1024)
30         if d:
31             data += d
32         else:
33             break
34     data = data.decode("utf8")
35     print(data)
36     client.close()
37
38 if __name__ == '__main__':
```

多线程，多进程编程



gil-global interpreter lock 全局解释器锁

python中一个线程对应于c语言中的一个线程

gil使得同一个时刻只有一个线程在一个cpu上执行字节码，无法将多个线程映射到多个cpu上执行。

gil会根据执行的字节码数以及时间片释放gil，gil在遇到io的操作时主动释放。

python反编译为字节码

```
1 import dis
2 def add(a):
3     a+=1
4     return a
5
6 print(dis.dis(add))
```

一个有线程安全问题的程序

```
1
2 import threading
3
4 total = 0
5
6 def add():
7     global total
8     for i in range(0,1000):
9         total += 1
10
11
12 def desc():
13     global total
14     for i in range(0,1000):
15         total -= 1
16
17 thread1 = threading.Thread(target=add)
18 thread2 = threading.Thread(target=desc)
19 thread1.start()
20 thread2.start()
21 thread1.join()
22 thread2.join()
23 print(total)
24
```

实现多线程

thread类实例化

```
1 #对于io操作来说 多线程和多进程性能差别不太
2 #1.通过Thread类实例化
3
4 import time
5 import threading
6
7 def get_detail_html(url):
8     print("get detail html started")
9     time.sleep(2)
10    print("get detail html end")
11
12 def get_detail_url(url):
13     print("get detail url started")
14     time.sleep(4)
15     print("get detail url end")
16
17 if __name__ == '__main__':
18     thread1 = threading.Thread(target=get_detail_html, args=("",))
19     thread2 = threading.Thread(target=get_detail_url, args=("",))
20     #thread1.setDaemon(True)
21     #thread2.setDaemon(True)
22     start_time=time.time()
23     thread1.start()
24     thread2.start()
25     thread1.join()
26     thread2.join()
27     print("last time:{}".format(time.time()-start_time))
28
```

继承thread类实现多线程

```
1 import threading
2 import time
3
4 class GetDetailHtml(threading.Thread):
5
6     def __init__(self,name):
7         super().__init__(name=name)
8
9     def run(self):
10         print("get detail html started")
11         time.sleep(2)
12         print("get detail html end")
13
14
15 class GetDetailUrl(threading.Thread):
16     def __init__(self,name):
17         super().__init__(name=name)
18
19     def run(self):
20         print("get detail url started")
21         time.sleep(4)
22         print("get detail url end")
23
24
25 if __name__ == '__main__':
26     thread1 = GetDetailHtml("get detail html")
27     thread2 = GetDetailUrl("get detail url")
28     start_time=time.time()
29     thread1.start()
30     thread2.start()
31     thread1.join()
32     thread2.join()
33     print("last time:{}".format(time.time()-start_time))
34
```

线程同步

Lock

```
1 import threading
2 from threading import Lock,RLock
3 #不可重入锁
4 #lock = Lock()
5 #可重入的锁
6 lock = RLock()
7 total = 0
8
9 def add():
10     global total
11     global lock
12     for i in range(1000000):
13         lock.acquire()
14         #lock.acquire()
15         total+=1
16         lock.release()
17         #lock.release()
18
19 def desc():
20     global total
21     global lock
22     for i in range(1000000):
23         lock.acquire()
24         total -= 1
25         lock.release()
26
27
28 if __name__ == '__main__':
29     thread1 = threading.Thread(target=add)
30     thread2 = threading.Thread(target=desc)
31     thread1.start()
32     thread2.start()
33     thread1.join()
34     thread2.join()
35     print(total)
```

condition

```
1 import threading
2 from threading import Condition
3
4 class XiaoAi(threading.Thread):
5     def __init__(self, cond):
6         super().__init__(name="小爱")
7         self.cond = cond
8
9     def run(self):
10        #调用with以后才可以调用:wai 和 notify方法。
11        with self.cond:
12            self.cond.wait()
13            print("{}:在".format(self.name))
14            self.cond.notify()
15            self.cond.wait()
16            print("{}:好啊".format(self.name))
17            self.cond.notify()
18            self.cond.wait()
19
20            print("{}:君住长江尾".format(self.name))
21            self.cond.notify()
22            self.cond.wait()
23
24            print("{}:共引长江水".format(self.name))
25            self.cond.notify()
26            self.cond.wait()
27
28
29            print("{}:次恨何时已".format(self.name))
30            self.cond.notify()
31            self.cond.wait()
32
33            print("{}:定不负相思意".format(self.name))
34            self.cond.notify()
35
36
37
```

```
38
39 class TianMao(threading.Thread):
40     def __init__(self, cond):
41         super().__init__(name="天猫精灵")
42         self.cond = cond
43
44     def run(self):
45         with self.cond:
46             print("{}:小爱同学".format(self.name))
47             self.cond.notify()
48             self.cond.wait()
49             print("{}:我们来对古诗吧".format(self.name))
50             self.cond.notify()
51             self.cond.wait()
52             print("{}:我住长江头".format(self.name))
53             self.cond.notify()
54             self.cond.wait()
55             print("{}:日日思君不见君".format(self.name))
56             self.cond.notify()
57             self.cond.wait()
58             print("{}:此水几时休".format(self.name))
59             self.cond.notify()
60             self.cond.wait()
61             print("{}:自愿君心似我心".format(self.name))
62             self.cond.notify()
63             self.cond.wait()
64
65 if __name__ == '__main__':
66     cond = threading.Condition()
67     xiaoai = XiaoAi(cond)
68     tianmao = TianMao(cond)
69     #注意：该语句需要先运行
70     # 肩动顺序很重要
71     # 在调用with cond之后才能调用wait或者notify方法
72     # condition有两层锁，一把底层锁会在线程调用了wait方法的时候释放，
73     # 上面的锁会在每次调用wait的时候分配一把并放入到cond的等待队列中，等到notify方法的唤醒。
74     xiaoai.start()
75     tianmao.start()
76     xiaoai.join()
```

```
77     tianmao.join()
```

```
78
```

semaphore

```
1  import threading
2  import time
3
4  class HtmlSpider(threading.Thread):
5      def __init__(self, url,sem):
6          super().__init__()
7          self.sem = sem
8
9      def run(self):
10         time.sleep(2)
11         print("get html text success")
12         self.sem.release()
13
14
15 class UrlProducer(threading.Thread):
16     def __init__(self, sem):
17         super().__init__()
18         self.sem = sem
19
20     def run(self):
21         for i in range(20):
22             self.sem.acquire()
23             html_thread = HtmlSpider(f"https://ww.baidu.com{i}",self.sem)
24             html_thread.start()
25
26 if __name__ == '__main__':
27     sem = threading.Semaphore(3)
28     url_producer = UrlProducer(sem)
29     url_producer.start()
30
```

线程池

```
1 import time
2 from concurrent.futures import ThreadPoolExecutor, as_completed, wait
3
4
5 #线程池，为什么要线程池
6 #主线程中可以获取某一个线程的状态或者某一个任务的状态，以及返回值
7 #当一个线程完成的时候我们主线程能立即知道
8 #futures 可以让多线程和多进程编码接口一致
9
10 def get_html(times):
11     time.sleep(times)
12     print("get page {} success".format(times))
13     return times
14
15 executor = ThreadPoolExecutor(max_workers=2)
16 #通过submit函数提交执行的函数到线程池中，submit 是立即返回
17 #task1 = executor.submit(get_html,(3))
18 #task2 = executor.submit(get_html,(2))
19 #done方法判断某个任务是否完成
20 #print(task1.done())
21 #cancel取消任务
22 #print(task1.cancel())
23 #result()获取任务结果
24 #print(task1.result())
25 #要获取已经成功的task的返回
26 urls = [3,2,4]
27 all_task =[executor.submit(get_html,(url)) for url in urls]
28 #等待所有任务执行完成
29 wait(all_task)
30 for future in as_completed(all_task):
31     data = future.result()
32     print("get {} page success".format(data))
33
34 #通过executor获取已经完成的task
35 #for data in executor.map(get_html,urls):
36 #    print("get {} page success".format(data))
```

多线程与多进程的区别

```
1 #耗cpu的操作，用多进程编程，对于io操作来说，使用多线程编程，进程切换代价要高于线程。
2 #1. 对于耗费cpu的操作，多进程优于多线程
3 import time
4 from concurrent.futures import ThreadPoolExecutor,as_completed,ProcessPoolExecutor
5
6 def fib(n):
7     if n<=2:
8         return 1
9     return fib(n-1)+fib(n-2)
10
11 def random_sleep(n):
12     time.sleep(n)
13     return n
14
15
16 def test_for_multi_thread(func,seq,testname):
17     with ThreadPoolExecutor(max_workers=3) as executor:
18         all_task = [executor.submit(func,(num)) for num in seq]
19         start_time = time.time()
20         for future in as_completed(all_task):
21             data = future.result()
22             #print("exe result: {}".format(data))
23         end_time = time.time()
24         cost = end_time - start_time
25         print(f"多线程测试{testname}time costed:{cost}")
26
27 def test_for_multi_process(func,seq,testname):
28     with ProcessPoolExecutor(max_workers=3) as executor:
29         all_task = [executor.submit(func,(num)) for num in seq]
30         start_time = time.time()
31         for future in as_completed(all_task):
32             data = future.result()
33             #print("exe result: {}".format(data))
34         end_time = time.time()
35         cost = end_time - start_time
36         print(f"多进程测试{testname}time costed:{cost}")
37
38
```

```
39
40 #对于IO操作来说，多线程优于多进程
41
42 if __name__ == '__main__':
43     test_for_multi_thread(fib, range(20, 35), "CPU计算密集型任务")
44     test_for_multi_process(fib, range(20, 35), "CPU计算密集型任务")
45     test_for_multi_thread(random_sleep, [2]*40, "IO密集型任务")
46     test_for_multi_process(random_sleep, [2]*40, "IO密集型任务")
```

多进程编程

fork

```
1 #该程序运行于Linux下
2 import os
3 import time
4 pid = os.fork()
5 print("bobby")
6 if pid == 0 :
7     print("子进程{},父进程: {}".format(str(os.getpid()),str(os.getppid()))
8 else:
9     print("我是父进程{}".format(str(pid)))
10 time.sleep(2)
```

multiprocessing

```
1 import multiprocessing
2 import time
3
4
5 def get_html(n):
6     time.sleep(n)
7     print("sub_process success")
8     return n
9
10
11 class MyProgress(multiprocessing.Process):
12
13     def __init__(self,n):
14         self.n=n
15         super().__init__()
16
17     def run(self):
18         time.sleep(self.n)
19         print("sub_process success")
20         return self.n
21
22
23 def test_multiprocessing_process():
24     progress = multiprocessing.Process(target=get_html,args=(2,))
25     print(progress.pid) #None
26     progress.start()
27     print(progress.pid)
28     progress.join()
29     print("main progress end")
30
31
32
33 def test_multiprocessing_process2():
34     myprocess=MyProgress(2)
35     myprocess.start()
36     myprocess.join()
37
```

```
38
39 def test_multiprocessing_process3():
40     pool = multiprocessing.Pool(multiprocessing.cpu_count())
41     result = pool.apply_async(get_html, args=(2,))
42     pool.close() #提交后一定要调改方法，否则会报错
43     pool.join()
44     print(result.get())
45
46
47
48 def test_multiprocessing_process4():
49     pool = multiprocessing.Pool(multiprocessing.cpu_count())
50     # for result in pool.imap(get_html, [1,5,3]):
51     #     print("{} sleep success".format(result))
52     for result in pool.imap_unordered(get_html, [1, 5, 3]):
53         print("{} sleep success".format(result))
54
55
56 if __name__ == '__main__':
57     # test_multiprocessing_process()
58     # test_multiprocessing_process2()
59     # test_multiprocessing_process3()
60     test_multiprocessing_process4()
61
```

进程间通讯

多进程间通讯不能用普通的Queue

```

1 import queue
2 import time
3 from multiprocessing import Process
4 from queue import Queue
5
6 from python高级.ch8.生成器读取大文件 import myradlines
7
8
9 def producer(queue):
10     queue.put("a")
11     time.sleep(1)
12
13 def consumer(queue):
14     time.sleep(2)
15     data = queue.get()
16     print(data)
17
18 if __name__ == '__main__':
19     queue = Queue(10)
20     my_producer = Process(target=producer, args=(queue,))
21     my_consumer = Process(target=consumer, args=(queue,))
22     my_producer.start()
23     my_consumer.start()
24     my_producer.join()
25     my_consumer.join()
26

```

以上代码会报错，应该改为如下写法

```
1 import time
2 from multiprocessing import Process,Queue
3
4 def producer(queue):
5     queue.put("a")
6     time.sleep(1)
7
8 def consumer(queue):
9     time.sleep(2)
10    data = queue.get()
11    print(data)
12
13 if __name__ == '__main__':
14     queue = Queue(10)
15     my_producer = Process(target=producer, args=(queue,))
16     my_consumer = Process(target=consumer, args=(queue,))
17     my_producer.start()
18     my_consumer.start()
19     my_producer.join()
20     my_consumer.join()
```

Manager

```
1 import time
2 from queue import Queue
3 from multiprocessing import Queue, Pool
4 from multiprocessing import Manager
5
6
7 #multiprocessing中的queue 不能用于pool进程池
8 #pool中的进程间通信需要使用manager 中的queue
9 def producer(queue):
10     queue.put("a")
11     time.sleep(2)
12
13 def consumer(queue):
14     time.sleep(2)
15     data = queue.get()
16     print(data)
17
18 if __name__ == '__main__':
19     queue = Manager().Queue()
20     pool = Pool(2)
21     pool.apply_async(producer, args=(queue,))
22     pool.apply_async(consumer, args=(queue,))
23     pool.close()
24     pool.join()
```

Pipe

```
1 import time
2 from multiprocessing import Pipe, Process
3
4
5 #multiprocessing中的queue 不能用于pool进程池
6 #pool中的进程间通信需要使用manager 中的queue
7 def producer(sendpipe):
8     sendpipe.send("bobby")
9     time.sleep(2)
10
11 def consumer(recvpipe):
12     time.sleep(2)
13     data=recvpipe.recv()
14     print(data)
15
16 if __name__ == '__main__':
17     receive_pipe, send_pipe = Pipe()
18     #pipe只能适用于两个指定的进程
19     #pipe的性能高于Queue
20     my_producer=Process(target=producer, args=(send_pipe,))
21     my_consumer=Process(target=consumer, args=(receive_pipe,))
22     my_producer.start()
23     my_consumer.start()
24     my_producer.join()
25     my_consumer.join()
```

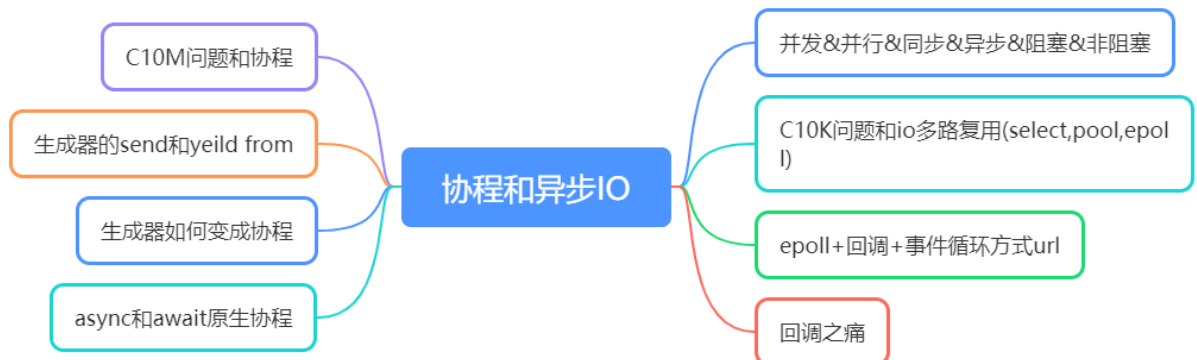
共享内存

```

1 from multiprocessing import Process, Manager
2
3 def add_data(p_dict, key, value):
4     p_dict[key] = value
5
6 if __name__ == '__main__':
7     progress_dict = Manager().dict()
8     first = Process(target=add_data, args=(progress_dict, 'key1', "bobby1"))
9     second = Process(target=add_data, args=(progress_dict, 'key2', "bobby2"))
10    first.start()
11    second.start()
12    first.join()
13    second.join()
14    print(progress_dict)

```

协程和异步IO



相关概念

并发&并行

并发是指一个时间段内) 有几个程序在同一个cpu上运行，但是任意时刻只有一个程序在cpu上运行
 并行是指任意时刻点上，有多个程序同时运行在多个cpu上

同步&异步

同步是指代码调用IO操作时 必须等待IO操作完成才返回的调用方式。

异步是指代码调用IO操作时，不必等IO操作完成就返回的调用方式。

阻塞&非阻塞

阻塞是指调用函数时候当前线程被挂起。

非阻塞是指调用函数时候当前线程不会被挂起，而是立即返回。

C10K问题和IO多路复用(select,poll,epoll)

C10k是一个在1999年被提出来的技术挑战

如何在一颗1GHz CPU， 2G内存， 1gbps网络环境下， 让单台服务器同时 为1万个客户端提供FTP服务

Unix下五种I/O模型

- 阻塞式I/O
- 非阻塞式I/O
- I/O复用
- 信号驱动式I/O
- 异步I/O(POSIX的aio _ 系列函数)

阻塞式IO

非阻塞式IO

IO复用

信号驱动式IO

异步IO

select,poll,epoll

- 1 **select**, **poll**, **epoll**都是IO多路复用的机制。I/O多路复用就是通过一种机制，一个进程可以监视多个描述符，一旦某个描述符就绪（一般是读就绪或者写就绪），能够通知程序进行相应的读写操作。但**select**, **poll**, **epoll**本质上都是同步I/O，因为他们都需要在读写事件就绪后自己负责进行读写，也就是说这个读写过程是阻塞的，而异步I/O则无需自己负责进行读写，异步I/O的实现会负责把数据从内核拷贝到用户空间。

select

- 1 **select** 函数监视的文件描述符分3类，分别是**writefds**、**readfds**、和**exceptfds**。调用后**select**函数会阻塞，直到有描述符就绪(有数据 可读、可写、或者有**except**)，或者超时(**timeout**指定等待时间，如果立即返回设为**null**即可)，函数返回。当**select**函数返回后，可以通过遍历**fdset**，来找到就绪的描述符。
- 2 **select**目前几乎在所有的平台上支持，其良好跨平台支持也是它的一个优点。**select**的一个缺点在于单个进程能够监视的文件描述符的数量存在最大限制，在Linux上一般为**1024**，可以通过修改宏定义甚至重新编译内核的方式提升这一限制，但是这样也会造成效率的降低。

poll

- 1 不同与**select**使用三个位图来表示三个**fdset**的方式，**poll**使用一个**pollfd**的指针实现。
- 2 **pollfd**结构包含了要监视的**event**和发生的**event**，不再使用**select**“参数-值”传递的方式。同时，**pollfd**并没有最大数量限制（但是数量过大后性能也是会下降）。和**select**函数一样，**poll**返回后，需要轮询**pollfd**来获取就绪的描述符。
- 3 从上面看，**select**和**poll**都需要在返回后，通过遍历文件描述符来获取已经就绪的**socket**。事实上，同时连接的大量客户端在一时刻可能只有很少的处于就绪状态，因此随着监视的描述符数量的增长，其效率也会线性下降

epoll

- 1 **epoll**是在2.6内核中提出的，是之前的**select**和**poll**的增强版本。相对于**select**和**poll**来说，**epoll**更加灵活，没有描述符限制。**epoll**使用一个文件描述符管理多个描述符，将用户关心的文件描述符的事件存放到内核的一个事件表中，这样在用户空间和内核空间的**copy**只需一次。
- 2
- 3 #1.**epoll**并不代表一定比**select**好。
- 4 #.在并发高的情况下，连接活跃度不是很高，**epoll**比**select**。
- 5 #.并发性不高，同时连接很活跃， **select**比**epoll**好。

select完成http请求

```

1 import socket
2 from selectors import DefaultSelector, EVENT_READ, EVENT_WRITE
3 from urllib.parse import urlparse
4
5
6 class Fetcher:
7     def __init__(self):
8         self.selector = DefaultSelector()
9         self.data = b""
10
11     def readable(self, key):
12         d = self.client.recv(1024)
13         if d:
14             self.data += d
15         else:
16             self.selector.unregister(key.fd)
17             data = self.data.decode("utf8")
18             html_data = data.split("\r\n\r\n")[1]
19             print(html_data)
20             self.client.close()
21
22     def connected(self, key):
23         self.selector.unregister(key.fd)
24         self.client.send("GET {} HTTP/1.1\r\nHost:
25 {} \r\nConnection:close\r\n\r\n".format(self.path, self.host).encode("utf8"))
26         self.selector.register(self.client.fileno(), EVENT_READ, self.readable)
27
28     def get_url(self, url):
29         url = urlparse(url)
30         self.port=80
31         self.path=url.path
32         if url.netloc.count(":")>0 :
33             seg=url.netloc.split(":")
34             self.host=seg[0]
35             self.port=int(seg[1])
36         else:
37             self.host=url.netloc

```

```

38         if self.path == "":
39             self.path = "/"
40
41         self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
42         #设置为非阻塞模式
43         self.client.setblocking(False)
44         try:
45             self.client.connect((self.host,self.port))
46         except BlockingIOError as e:
47             pass
48         #注册
49         self.selector.register(self.client.fileno(),EVENT_WRITE,self.connected)
50
51     def loop(self):
52         while True:
53             ready = self.selector.select()
54             for key,mask in ready:
55                 call_back = key.data
56                 call_back(key)
57
58
59 if __name__ == '__main__':
60     f = Fetcher()
61     f.get_url("http://www.baidu.com/")
62     f.loop()

```

协程

C10M问题

如何利用8核心CPU，64G内存，在10gbps的网络上保持1000万并发连接。

1. 回调模式编码复杂度高。
2. 同步编程的并发性不高。
3. 多线程编程需要线程间同步，lock。

1. 采用同步的方式去编写异步的代码。
2. 使用单线程去切换任务：

- 2.1. 线程是由操作系统切换的，单线程切换意味着我们需要程序员自己去调度任务。
- 2.2. 不在需要锁，并发性高，如果单线程内切换函数，性能远高于线程切换，并发性更高。

协程与生成器

案例1

```
1 def gen_func():
2     html = yield "http://www.baidu.com"
3     print(html)
4     yield 2
5     yield 3
6     return "bobby"
7
8
9 def test_gen_001():
10    gen = gen_func()
11    # 通过next启动生成器
12    # 在调用send发送非none值之前，我们必须启动一次生成器，方式有两种
13    # 1.gen.send(None)。2:gen.next()
14    url = next(gen)
15    print(url)
16    html = "bobby"
17    # 通过send启动生成器
18    url = gen.send(html)
19    print(url)
20    url = next(gen)
21    print(url)
22
23
24 if __name__ == '__main__':
25     test_gen_001()
```

throw

```

1  def gen_func():
2      try:
3          yield "http://www.baidu.com"
4      except Exception as e:
5          pass
6      yield 2
7      yield 3
8      return "bobby"
9
10 if __name__ == '__main__':
11     gen = gen_func()
12     print(next(gen))
13     gen.throw(Exception, "download error")
14     print(next(gen))
15     gen.throw(Exception, "download error")
16

```

close

```

1  def gen_func():
2      try:
3          yield "http://www.baidu.com"
4      except GeneratorExit:
5          pass
6
7      yield 2
8      yield 3
9      return "bobby"
10
11 if __name__ == '__main__':
12     gen = gen_func()
13     print(next(gen))
14     gen.close()
15     next(gen)

```

